

eqrcp局域网文件传输系统 软件设计说明书

软件名称: eqrcp局域网文件传输系统

软件版本: V1.0

编制日期: 2026年5月

开发语言: Go 1.26.2、JavaScript、HTML5、CSS3

1. 引言

1.1 编写目的

本文档详细描述eqrcp局域网文件传输系统的软件架构、功能模块、技术实现和设计思路，为软件著作权登记提供技术说明，同时为后续维护和升级提供参考。

1.2 项目背景

在日常工作和生活中，用户经常需要在计算机和移动设备之间传输文件。传统方法如数据线连接、云存储上传下载、即时通讯工具传输等，存在操作繁琐、需要网络流量、传输速度慢等问题。eqrcp通过局域网直连方式，利用二维码快速建立连接，实现高速、便捷、安全的文件传输。

1.3 术语定义

局域网 (LAN) : Local Area Network, 本地区域网络

二维码 (QR Code) : Quick Response Code, 快速响应码

HTTP/HTTPS: 超文本传输协议及其安全版本

WebSocket: 基于TCP的全双工通信协议

TLS: Transport Layer Security, 传输层安全协议

CLI: Command Line Interface, 命令行接口

GUI: Graphical User Interface, 图形用户界面

1.4 参考资料

Go语言官方文档: <https://go.dev/doc/>

HTTP/1.1 RFC 7230-7235

WebSocket协议 RFC 6455

QR Code标准 ISO/IEC 18004

2. 系统概述

2.1 系统目标

eqrcp旨在提供一个简单、快速、安全的局域网文件传输解决方案，主要目标包括：

1. 零配置：无需复杂设置，即开即用
2. 跨平台：支持Windows、Linux、 macOS
3. 高性能：利用局域网带宽实现高速传输
4. 易用性：通过二维码扫描快速连接
5. 安全性：支持HTTPS加密传输
6. 多功能：支持文件发送、接收、实时聊天

2.2 系统特点

1. **命令行优先设计**: 通过简洁的命令行接口快速启动服务
2. **自动网络配置**: 自动检测可用网络接口和IP地址
3. **二维码连接**: 在终端显示二维码，移动设备扫码即可访问
4. **内置Web服务器**: 无需外部Web服务器，独立运行
5. **桌面集成**: Windows系统支持右键菜单快速分享
6. **实时通信**: 基于WebSocket的实时聊天功能
7. **灵活配置**: 支持配置文件、环境变量、命令行参数多种配置方式

2.3 应用场景

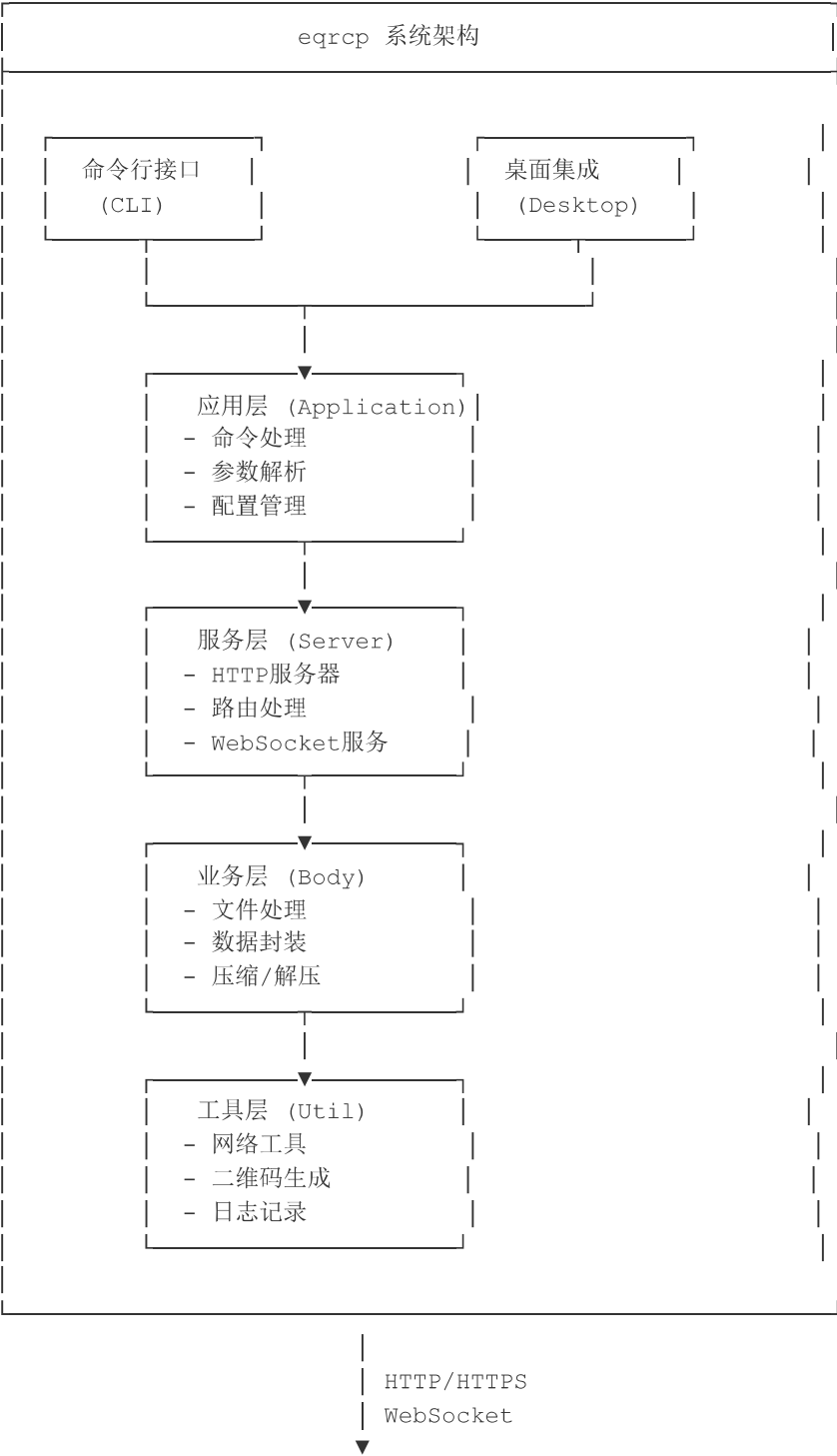
1. 手机拍摄的照片快速传输到电脑
2. 电脑文档发送到手机查看
3. 团队成员间局域网内快速共享文件
4. 临时文本内容在设备间传递

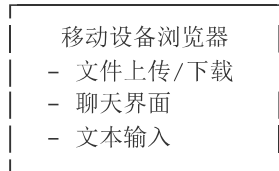
5. 局域网内多人实时聊天和文件共享

3. 系统架构

3.1 总体架构

eqrcp采用客户端-服务器（C/S）架构，计算机端运行服务器程序，移动设备通过浏览器作为客户端访问。





3.2 技术架构

后端框架: Go标准库 (net/http)

前端技术: 原生JavaScript + HTML5 + CSS3

通信协议: HTTP/HTTPS、WebSocket

数据格式: JSON、Multipart Form Data

加密技术: TLS 1.2+

二维码: go-qrcode库

命令行: Cobra框架

配置管理: Viper库

3.3 模块划分

系统主要分为以下模块：

1. **命令行接口模块 (cmd)** : 处理用户命令和参数
2. **应用层模块 (application)** : 应用逻辑和流程控制
3. **服务器模块 (server)** : HTTP服务和WebSocket服务
4. **文件处理模块 (body)** : 文件读取、压缩、传输
5. **配置模块 (config)** : 配置文件读写和管理
6. **二维码模块 (qr)** : 二维码生成和显示
7. **工具模块 (util)** : 网络、日志等通用工具
8. **桌面集成模块 (desktop)** : 操作系统集成功能
9. **日志模块 (logger)** : 日志记录和输出
10. **版本模块 (version)** : 版本信息管理

4. 功能模块设计

4.1 命令行接口模块

4.1.1 模块功能

提供用户交互的命令行接口，解析用户输入的命令和参数，调用相应的功能模块。

4.1.2 主要命令

1. **发送文件:** `eqrcp <文件路径>`
2. **接收文件:** `eqrcp receive`
3. **聊天模式:** `eqrcp chat`
4. **配置向导:** `eqrcp config`
5. **桌面集成:** `eqrcp desktop [install|uninstall|status]`
6. **版本信息:** `eqrcp version`
7. **帮助信息:** `eqrcp help`

4.1.3 参数支持

`--interface, -i` : 指定网络接口
`--port, -p` : 指定服务端口
`--path` : 指定URL路径
`--output, -o` : 指定接收文件保存目录
`--secure, -s` : 启用HTTPS
`--tls-cert` : TLS证书路径
`--tls-key` : TLS私钥路径
`--browser, -b` : 在浏览器中打开
`--keep-alive, -k` : 保持服务运行
`--zip` : 强制使用zip压缩

4.1.4 技术实现

使用Cobra框架构建命令行接口，支持子命令、标志参数、环境变量等多种输入方式。

4.2 文件发送模块

4.2.1 模块功能

将计算机上的文件通过HTTP服务提供给移动设备下载。

4.2.2 处理流程

1. 用户指定要发送的文件或目录
2. 系统检查文件是否存在和可读
3. 如果是多个文件或目录，自动创建临时zip压缩包
4. 启动HTTP服务器，提供文件下载接口
5. 生成访问URL和二维码
6. 在终端显示二维码
7. 移动设备扫码访问并下载文件
8. 下载完成后关闭服务（或根据--keep-alive参数保持运行）

4.2.3 技术细节

单文件直接提供下载

多文件/目录使用zip格式压缩

支持断点续传（HTTP Range请求）

自动设置Content-Type和Content-Disposition头

临时文件在传输完成后自动清理

4.3 文件接收模块

4.3.1 模块功能

启动HTTP服务器，提供文件上传界面，接收移动设备上传的文件。

4.3.2 处理流程

1. 用户执行 `eqrcp receive` 命令
2. 系统启动HTTP服务器

3. 生成上传页面URL和二维码
4. 移动设备扫码访问上传页面
5. 用户选择文件并上传
6. 服务器接收文件并保存到指定目录
7. 显示上传成功信息

4.3.3 支持的上传类型

文件上传（支持多文件）
文本粘贴（保存为.txt文件）
剪贴板文件（支持的浏览器）
剪贴板图片（保存为.png文件）

4.3.4 技术实现

使用multipart/form-data处理文件上传
支持大文件分块上传
文件名冲突自动重命名
上传进度实时显示

4.4 实时聊天模块

4.4.1 模块功能

提供基于WebSocket的实时聊天功能，支持文本消息和文件附件。

4.4.2 功能特性

1. 实时双向消息传输
2. 支持文本消息
3. 支持文件附件发送
4. 消息历史记录
5. 多设备同时连接
6. 连接状态显示

7. 设备主题颜色标识

4.4.3 技术实现

使用WebSocket协议实现全双工通信

消息格式采用JSON

文件通过Base64编码传输或单独HTTP上传

服务端维护连接池管理多个客户端

心跳机制检测连接状态

4.4.4 消息类型

```
text : 文本消息
file : 文件附件
system : 系统通知
ping/pong : 心跳消息
```

4.5 配置管理模块

4.5.1 模块功能

管理软件配置，支持配置文件、环境变量、命令行参数三种配置方式。

4.5.2 配置优先级

命令行参数 > 环境变量 > 配置文件 > 默认值

4.5.3 配置文件位置

Linux/macOS: ~/.config/eqrcp/config.yml

Windows: %APPDATA%\eqrcp\config.yml

4.5.4 主要配置项

```
interface: any      # 网络接口
port: 0             # 端口（0表示随机）
path: ""            # URL路径（空表示随机）
fqdn: ""            # 完全限定域名
keepalive: false    # 保持服务运行
secure: false        # 启用HTTPS
tls-cert: ""         # TLS证书路径
tls-key: ""          # TLS私钥路径
output: "."          # 接收文件保存目录
```

4.5.5 环境变量

所有配置项都支持通过 `EQRCP_` 前缀的环境变量设置，例如：

```
EQRCP_INTERFACE=wlan0
EQRCP_PORT=8080
EQRCP_KEEPAIVE=true
```

4.6 网络模块

4.6.1 模块功能

处理网络相关功能，包括网络接口检测、IP地址获取、端口分配等。

4.6.2 主要功能

1. **网络接口检测**: 自动检测可用的网络接口
2. **IP地址获取**: 获取本机局域网IP地址
3. **外网IP检测**: 可选的外网IP获取（用于FQDN场景）
4. **端口分配**: 自动分配可用端口或使用指定端口
5. **URL生成**: 根据IP、端口、路径生成访问URL

4.6.3 网络接口选择策略

1. 如果指定了接口名称，使用指定接口
2. 如果指定了 `any`，自动选择第一个可用接口
3. 如果未指定，提示用户选择

4.7 二维码模块

4.7.1 模块功能

生成二维码并在终端显示，方便移动设备扫描。

4.7.2 技术实现

使用go-qrcode库生成二维码

将二维码转换为ASCII字符在终端显示

支持正常和反色两种显示模式
自动调整二维码大小适应终端宽度

4.7.3 显示方式

1. 终端ASCII显示（默认）
2. 浏览器显示（--browser参数）
3. 同时显示URL文本

4.8 桌面集成模块

4.8.1 模块功能

将eqrcp集成到操作系统桌面环境，提供右键菜单等快捷操作。

4.8.2 Windows集成

1. **右键菜单**: 文件/文件夹右键菜单添加"Share with eqrcp"
2. **发送到菜单**: 添加到"发送到"菜单
3. **多文件选择**: 支持选择多个文件一起分享
4. **注册表管理**: 自动管理Windows注册表项

4.8.3 桌面命令

```
eqrcp desktop install : 安装桌面集成  
eqrcp desktop uninstall : 卸载桌面集成  
eqrcp desktop status : 查看集成状态  
eqrcp desktop share <文件> : 分享文件  
eqrcp desktop receive <目录> : 接收文件到指定目录  
eqrcp desktop chat : 启动聊天
```

4.8.4 启动器程序

提供 eqrcp-launcher.exe ，用于在Windows资源管理器中启动eqrcp并显示GUI窗口。

4.9 安全模块

4.9.1 HTTPS支持

支持自定义TLS证书和私钥
自动生成自签名证书（开发模式）
TLS 1.2及以上版本

4.9.2 安全特性

1. 仅监听局域网地址，不暴露到公网
2. 随机URL路径防止猜测
3. 传输完成后自动关闭服务
4. 临时文件自动清理
5. 文件名安全过滤

5. 数据结构设计

5.1 配置数据结构

```
type Config struct {
    Interface string // 网络接口
    Port      int    // 端口
    Path      string // URL路径
    FQDN      string // 域名
    KeepAlive bool   // 保持运行
    Secure     bool   // HTTPS
    TLSCert   string // 证书路径
    TLSKey    string // 私钥路径
    Output     string // 输出目录
    Bind      string // 绑定地址
    Browser    bool   // 打开浏览器
    Reversed   bool   // 反色二维码
    Zip       bool   // 强制zip
}
```

5.2 文件传输数据结构

```
type Payload struct {
    Path          string // 文件路径
    IsArchive     bool   // 是否压缩包
    DeleteAfter    bool   // 传输后删除
    ArchiveName   string // 压缩包名称
}
```

5.3 聊天消息数据结构

```
{
    "type": "text|file|system",
    "sender": "device-id",
    "content": "消息内容",
    "timestamp": 1234567890,
    "filename": "文件名（仅file类型）",
    "filesize": 12345,
    "filedata": "base64编码数据"
}
```

5.4 网络信息数据结构

```
type NetworkInfo struct {
    InterfaceName string // 接口名称
    IPAddress     string // IP地址
}
```

```
    Port      int      // 端口
    URL        string   // 完整URL
}
```

6. 接口设计

6.1 HTTP接口

6.1.1 文件下载接口

路径: GET `{random-path}`

功能: 下载文件

响应: 文件流

头部: Content-Disposition, Content-Type, Content-Length

6.1.2 文件上传接口

路径: POST `{random-path}/upload`

功能: 上传文件

请求: multipart/form-data

响应: JSON状态

6.1.3 上传页面接口

路径: GET `{random-path}`

功能: 显示上传页面

响应: HTML页面

6.1.4 聊天页面接口

路径: GET `{random-path}`

功能: 显示聊天页面

响应: HTML页面

6.2 WebSocket接口

6.2.1 聊天连接

路径: WS `{random-path}/ws`

功能: 建立WebSocket连接

消息格式: JSON

6.2.2 消息类型

1. **文本消息:** {"type":"text","content":"..."}
2. **文件消息:** {"type":"file","filename":"...","filedata":"..."}
3. **系统消息:** {"type":"system","content":"..."}
4. **心跳消息:** {"type":"ping"} / {"type":"pong"}

7. 关键技术实现

7.1 自动网络配置

系统启动时自动检测可用网络接口，选择合适的IP地址和端口，无需用户手动配置。

7.2 二维码快速连接

将访问URL编码为二维码，在终端显示，移动设备扫码即可访问，省去手动输入URL的麻烦。

7.3 临时文件管理

多文件传输时自动创建临时zip压缩包，传输完成后自动清理，不占用磁盘空间。

7.4 WebSocket实时通信

使用WebSocket协议实现全双工通信，消息实时推送，无需轮询，降低延迟和服务器负载。

7.5 跨平台支持

使用Go语言开发，天然支持跨平台编译，一套代码可编译为Windows、Linux、macOS可执行文件。

7.6 桌面集成

通过操作系统API和注册表操作，将eqrcp集成到系统右键菜单，提供原生体验。

8. 性能设计

8.1 传输性能

利用局域网带宽，传输速度可达100Mbps+
支持HTTP Range请求，实现断点续传
大文件分块传输，避免内存溢出

8.2 并发处理

HTTP服务器支持多并发连接
WebSocket连接池管理多个客户端
Go协程处理并发请求，高效利用CPU

8.3 资源占用

单进程运行，内存占用小于50MB
临时文件及时清理，不占用磁盘空间
空闲时CPU占用接近0%

9. 安全设计

9.1 网络安全

仅监听局域网地址，不暴露到公网
支持HTTPS加密传输
随机URL路径，防止未授权访问

9.2 文件安全

文件名安全过滤，防止路径遍历攻击
上传文件大小限制
临时文件权限控制

9.3 隐私保护

不收集用户数据
不连接外部服务器
所有数据在局域网内传输

10. 可扩展性设计

10.1 模块化设计

各功能模块独立，便于维护和扩展。

10.2 配置灵活性

支持配置文件、环境变量、命令行参数多种配置方式，适应不同使用场景。

10.3 插件机制

预留接口，未来可支持插件扩展功能。

11. 测试设计

11.1 单元测试

对各功能模块编写单元测试，确保代码质量。

11.2 集成测试

测试模块间协作，验证整体功能。

11.3 跨平台测试

在Windows、Linux、macOS上测试，确保跨平台兼容性。

12. 部署说明

12.1 编译

```
go build -o eqrcp .
```

12.2 安装

将编译后的可执行文件放到系统PATH目录即可。

12.3 配置

首次运行执行 `eqrcp config` 进行配置向导。

13. 维护说明

13.1 日志

系统运行日志记录在标准输出，可重定向到文件。

13.2 错误处理

所有错误都有明确的错误信息，便于排查问题。

13.3 版本管理

使用语义化版本号，主版本号.次版本号.修订号。

14. 总结

eqrcp局域网文件传输系统采用模块化设计，功能完善，性能优异，安全可靠。通过命令行接口和桌面集成，为用户提供便捷的文件传输体验。系统采用Go语言开发，具有良好的跨平台性和可维护性，为后续功能扩展奠定了坚实基础。